

ROB301- FINAL PROJECT REPORT

By Riya Mathur (1008747867), Manan Sawrangpate (1008770386)

Introduction

The Galbraith Memorial Mail Robot project simulated a real-world autonomous robotics application by designing a control system for the Turtlebot 3 Waffle Pi. The robot navigated a closed-loop map, identified office locations using coloured floor patches, and delivered mail to three randomly assigned offices starting from an arbitrary location. Navigation relied on white tape as a guide and the onboard camera for colour detection and line-following. To resolve ambiguities from multiple offices sharing the same colour, Bayesian localization was employed to probabilistically determine the robot's position. This project followed an incremental approach, progressing from basic navigation to a proof-of-concept demonstration, emphasizing the integration of hardware and advanced algorithms to address challenges in localization, navigation, and autonomous task execution.

Brief Description of Robot Platform

The Turtlebot 3 Waffle Pi is a compact and versatile robotic platform equipped with hardware and software components tailored for navigation and autonomous tasks. Its key hardware feature is an onboard camera, which serves a dual purpose: detecting coloured patches on the floor to identify office locations and tracking the white tape used as a guiding path. The camera outputs two key metrics: the mean RGB values for colour detection and line index data indicating the position of the white tape. In addition to the camera, the robot utilizes wheel encoders to measure movement and track its position along the path. These sensors, combined with the camera, enable the implementation of Bayesian localization and PID control to achieve precise navigation, localization, and task execution for the mail delivery simulation. The software framework, built on ROS, facilitated seamless integration of the robot's sensors and control algorithms.

Overview of Solution Strategy

The solution strategy combined line-following control from lab 3 with Bayesian localization to enable the robot to estimate its location and map the closed-loop route while navigating autonomously and delivering mail to target offices. The robot followed the white tape on the floor using a PID controller, ensuring smooth and accurate path traversal. Simultaneously, Bayesian localization was used to estimate the robot's position by merging sensor data from the robot camera with a predefined state model given in project handout. The localization process used probabilistic reasoning to update the robot's belief about its location based on observed floor colours and movement commands. The strategy included stopping at target offices upon identification, simulating mail delivery, and resuming line-following navigation. By dividing the system into modular components—line-following, localization, and task execution—the approach aimed to achieve robust and adaptive behaviour despite challenges such as sensor noise and environmental inconsistencies.

Technical Details on the Design Methodology

The technical implementation of the Galbraith Memorial Mail Robot project was structured around two core subsystems: motion control for path following and probabilistic localization for determining the robot's position. Both subsystems were integrated to ensure that the robot could reliably navigate the route, identify office locations, and deliver mail.

1. Motion Control

PID Control for Line Following:

The robot used a PID (Proportional-Integral-Derivative) controller to follow the white tape path. The camera's line index data provided RGB values which were compared to a predefined set of RGB values. When the RGB values most closely matched with the colour white, it indicated the position of the tape relative to the robot's field of view. This discrepancy served as the error input to the PID controller which then adjusted the robot's movement to minimize deviation from the line. The control process involved:

- **Proportional Term:** Calculated corrections based on the magnitude of the error to minimize deviation from the line.
- **Integral Term:** Accumulated past errors to address drift over time and ensure consistent corrections.
- **Derivative Term:** Predicted future errors by analyzing the rate of change of the error, allowing smoother adjustments.

The PID controller's outputs were applied to the robot's angular velocity to keep it aligned with the tape, while a constant linear velocity ensured forward motion. Parameters such as k_P , k_I , and k_D were tuned experimentally to balance responsiveness and stability, while minimizing oscillations and overcorrections.

2. Localization Using Bayesian Estimation

State Model: The robot's environment was represented as a discrete topological map of office locations connected by paths. A state model was defined to describe the probabilities of transitioning between states (offices) based on movement commands (u):

- **Move Left ($u = -1$):** High probability of transitioning to the previous state, with smaller probabilities of staying or moving forward.
- **Stay ($u = 0$):** High probability of remaining in the current state, with small probabilities of moving left or right.
- **Move Right ($u = +1$):** High probability of transitioning to the next state, with smaller probabilities of staying or moving left.

These probabilities were encoded in a transition matrix, which was used to predict the robot's position based on the control input. The results were then stored in a prediction array.

Measurement Model: The measurement model defined the probability of observing a specific floor colour given the robot's state. The RGB values detected by the camera were matched to predefined table of colour codes using Euclidean distance. Observed colours were mapped to the most likely colour using the measurement model, which accounted for sensor noise and environmental factors (e.g., lighting conditions).

Bayesian Update: The robot combined the state prediction and measurement update to calculate the posterior probabilities of being in each state:

1. **Prediction Step:** Multiplied the current belief (prior probabilities) by the transition matrix to compute predicted probabilities (this calls the state prediction function).

2. **Update Step:** Multiplied the predicted probabilities by the measurement likelihoods for the observed colour. The result was normalized to ensure the probabilities summed to 1. (this calls the measurement model function).

This recursive process allowed the robot to refine its position estimate as it moved and observed colours along the route.

3. Task Execution

Identifying Offices: When the robot passed a coloured patch, it compared the observed colour to its predefined map to determine whether it was at an office location. Bayesian localization resolved ambiguities caused by identical colours representing different offices, requiring the robot to pass several patches before confidently estimating its position.

Delivering Mail: Upon identifying a target office, the robot executed a function called “stop_at_office()” which stopped the robot in place and performed a 90° rotation to simulate mail delivery. This function also had the robot move forwards for a set amount of time so that it would be able to clear the office before subsequently resuming following the tape. Lastly, delivered offices were removed from the target list, and the robot continued until all mail was delivered.

4. Integration and High-Level Logic

Main Loop: The system operated in a loop where the robot:

1. Followed the line using PID control.
2. Processed camera data to update its position using Bayesian localization.
3. Checked whether the current state matched a target office.
4. Executed the delivery routine if at a target office.
5. Ended when all packages delivered, outputting the final state probabilities.

A specific function would input the RGB values the camera detects, storing in a variable that can later be accessed to determine whether the robot should move forward or if the robot is at an office location and should perform task execution.

Error Handling: To account for potential errors, such as the robot losing sight of the tape or receiving ambiguous colour readings:

- The line-following controller incorporated logic to reorient the robot if it deviated from the path.
- Calibration of the colour sensors was also done to ensure that the RGB values detected rather than expected were used to estimate position.
- A fallback mechanism defaulted to a uniform probability distribution when the localization confidence dropped, allowing recovery over subsequent iterations.
- After detecting an office, the robot moved forward for a specific period to pass the office and resume following the line.

Summary of demonstration performance

Our demonstration was structured around the deliverables outlined in the project handout. Despite significant effort and confidence in our theoretical approach, we faced challenges that prevented us

from successfully completing the tasks. These challenges included hardware connection issues, delays due to outdated starter code, and difficulties in integrating various system components.

For Deliverable 2: Full-Route Execution, our goal was to demonstrate the robot's ability to traverse a practice course, following the white line and stopping at each door along the route. Unfortunately, we were unable to achieve a fully functional system due to intermittent connectivity issues between the robot and the remote PC. These issues limited our testing time and made it difficult to debug the line-following behaviour. While our code logic for detecting and following the white line was sound (as mentioned before, it was unexpected as we used most of our lab-3 code which worked properly before), the lack of real-world validation hindered progress.

For Deliverable 3: Bayesian-Localization Node, we aimed to implement and demonstrate the robot's ability to probabilistically estimate its location by passing a series of offices. While we developed a functioning Bayesian localization algorithm in isolation, the inability to integrate it with the robot's movement system prevented us from testing this functionality. Additionally, the hardware delays further constrained the time needed to refine this deliverable.

Finally, for Deliverable 4: Proof-of-Concept Demonstration, the objective was to integrate all components into a complete control system capable of executing a full test course. During this phase, our inability to integrate individual modules became most apparent. Each component, including line following, Bayesian localization, and navigation, worked independently during preliminary tests but failed when combined due to compatibility issues and conflicts in data flow. Limited testing time before the demonstration compounded these problems, leaving us unable to achieve a working system during the final presentation.

Despite these setbacks, the project provided valuable lessons. We gained insight into the importance of robust testing, early detection of compatibility issues, and effective communication within the team. These experiences will inform our approach to future projects, allowing us to better anticipate and address similar challenges.

List of Potential Improvements for the Future

1. **Robust Testing Protocols:** Conduct thorough hardware and connectivity testing at the start of the project to identify and resolve issues before they impact progress. Implement incremental testing for each deliverable, ensuring that each module functions as expected before moving on to the next.
2. **Incremental System Integration:** Develop and test modules incrementally, integrating smaller subsystems as they are completed. This would help identify compatibility issues early in the process and avoid last-minute integration challenges.
3. **Early Resource Verification:** Verify the compatibility of starter code and hardware with the project setup at the beginning to avoid delays caused by outdated or incompatible resources.
4. **Time Management and Contingency Planning:** Allocate additional time for testing and debugging, particularly for the final integration phase. Develop contingency plans for

By implementing these improvements, we aim to ensure smoother progress and better outcomes in future projects, minimizing delays and increasing the likelihood of success during demonstrations.

Conclusion

To conclude, this project aimed to simulate real-world autonomous robot systems by integrating a PID control algorithm with Bayesian localization to achieve probabilistic

estimation of the robot's position. Throughout the process of building, calibrating and testing our solution, we learned how to handle sensor error, undefined behaviour, and troubleshooting complex algorithms. As outlined in our future improvements sections, this project outlined the importance of incremental complexity, establishing effective debugging procedures and testing the integration between hardware and software components. While we were not able to meet the deliverables, the project provided us with practical insights we can bring into our future projects. These lessons, particularly in system integration, debugging, and incremental complexity, will help us hone our skillset and improve the outcomes of our future work in autonomous robotics.

Appendix

```
#!/usr/bin/env python
import rospy
import math
from geometry_msgs.msg import Twist
from std_msgs.msg import String, UInt32MultiArray, Float64MultiArray, UInt32
import numpy as np
import colorsys

class BayesLoc:
    def __init__(self, p0, colour_codes, colour_map):
        self.colour_sub = rospy.Subscriber(
            "mean_img_rgb", Float64MultiArray, self.colour_callback
        )
        self.line_sub = rospy.Subscriber("line_idx", UInt32, self.line_callback)
        self.cmd_pub = rospy.Publisher("cmd_vel", Twist, queue_size=1)

        self.num_states = len(p0)
        self.colour_codes = colour_codes
        self.colour_map = colour_map
        self.probability = p0
        self.state_prediction = np.zeros(self.num_states)
        self.posterior_probs = 0
        self.cur_colour = None # most recent measured colour
        self.current_twist = 0
        self.error = 0

    def colour_callback(self, msg):
        """
        callback function that receives the most recent colour measurement from the camera.
        """
        self.cur_colour = np.array(msg.data) # [r, g, b]
        print(self.cur_colour)

    def line_callback(self, msg):
```

```
"""
```

```
Handles line index updates. Processes input and computes motion control.
```

```
"""
```

```
self.error = 350 - msg.data
```

```
pass
```

```
def follow_line(self):
```

```
    #follow the line
```

```
    rate = rospy.Rate(100)
```

```
    twist = Twist()
```

```
    twist.linear.x = 0.6
```

```
    twist.angular.z = 0
```

```
    kP = 0.004
```

```
    kI = 0.00005
```

```
    kD = 0.003
```

```
    acc_error = 0
```

```
    last_error = self.error
```

```
    filtered_error = 0
```

```
    filtered_error = self.error# * 0.3 + filtered_error * 0.7
```

```
    acc_error += filtered_error
```

```
    diff_error = filtered_error - last_error
```

```
    twist.angular.z = kP * filtered_error + kI * acc_error + kD * diff_error
```

```
    self.current_twist = twist.angular.z
```

```
    twist.linear.x = 0.1
```

```
    last_error = filtered_error
```

```
self.cmd_pub.publish(twist)
```

```
print("Linear: " + str(twist.linear.x), "angular: " + str(twist.angular.z))
```

```
return
```

```
def wait_for_colour(self):
```

```
    """Loop until a colour is received."""
```

```
    rate = rospy.Rate(100)
```

```
    while not rospy.is_shutdown() and self.cur_colour is None:
```

```
        rate.sleep()
```

```
def stop_at_office(self):
```

```
    twist = Twist()
```

```
    twist.linear.x = 0
```

```
    self.cmd_pub.publish(twist)
```

```
    rospy.sleep(3) # Pause for 3 seconds
```

```
    # Rotate 90 degrees left
```

```
    twist.angular.z = math.radians(90)
```

```
    self.cmd_pub.publish(twist)
```

```
rospy.sleep(2)
```

```
# Rotate back to original orientation
twist.angular.z = -math.radians(90)
self.cmd_pub.publish(twist)
rospy.sleep(2)
```

```
# Move forward to par the office
twist.linear.x = 1.5
self.cmd_pub.publish(twist)
rospy.sleep(4)
```

```
def state_model(self, u):
```

```
    N = self.num_states
```

```
    # Initialize empty transition matrix
    transition_matrix = np.zeros((N, N))
```

```
    # Define the probabilities based on action `u`
```

```
    if u == -1: # Move Left
```

```
        for i in range(N):
```

```
            if i > 0:
```

```
                transition_matrix[i, i - 1] = 0.85 # Transition left
```

```
                transition_matrix[i, i] = 0.10      # Stay
```

```
            if i < N - 1:
```

```
                transition_matrix[i, i + 1] = 0.05 # Transition right
```

```
    elif u == 0: # Stay
```

```
        for i in range(N):
```

```
            if i > 0:
```

```
                transition_matrix[i, i - 1] = 0.05 # Transition left
```

```
                transition_matrix[i, i] = 0.90      # Stay
```

```
            if i < N - 1:
```

```
                transition_matrix[i, i + 1] = 0.05 # Transition right
```

```
    elif u == +1: # Move Right
```

```
        for i in range(N):
```

```
            if i > 0:
```

```
                transition_matrix[i, i - 1] = 0.05 # Transition left
```

```
                transition_matrix[i, i] = 0.10      # Stay
```

```
            if i < N - 1:
```

```
                transition_matrix[i, i + 1] = 0.85 # Transition right
```

```

# Compute the predicted state probabilities:  $p(x_{k+1}) = T * p(x_k)$ 
self.state_prediction = np.dot(transition_matrix, self.probability)
rospy.loginfo(f"State prediction: {self.state_prediction}")

```

```

def measurement_model(self, x):

```

```

    if self.cur_colour is None:
        self.wait_for_colour()

```

```

    prob = np.zeros(len(self.colour_codes))

```

```

    measurement_probs = {
        'blue': [0.60, 0.20, 0.05, 0.05],
        'green': [0.20, 0.60, 0.05, 0.05],
        'yellow': [0.05, 0.05, 0.65, 0.20],
        'orange': [0.05, 0.05, 0.15, 0.60],
        'nothing': [0.10, 0.10, 0.10, 0.10]
    }

```

```

    colour_names = ['blue','green','yellow','orange','nothing']

```

```

    # Find the observed color (cur_colour) by calculating the closest match

```

```

    min_distance = float('inf')
    observed_color = 'nothing'
    for idx, ref_colour in enumerate(self.colour_codes):
        distance = np.linalg.norm(self.cur_colour - ref_colour)
        if distance < min_distance:
            min_distance = distance
            observed_color = colour_names[idx]

```

```

    rospy.loginfo(f"Observed color: {observed_color}")

```

```

    # Assign probabilities based on the measurement model for the observed color

```

```

    prob = np.array(measurement_probs[observed_color])
    rospy.loginfo(f"Shape of measurement_probs: {prob.shape}")

```

```

    return prob

```

```

def state_predict(self):

```

```

    rospy.loginfo("predicting state")
    """

```

```

    TODO: Complete the state prediction function: update
    self.state_prediction with the predicted probability of being at each
    state (office)

```



```

"""
#step 1 check if control is R or L
twist_angular = self.current_twist #check this
if twist_angular > 0:
    u = 1
elif twist_angular == 0:
    u = 0
else:
    u = -1
self.state_model(u)
#Use the measurement model to get measurement probabilities
#measurement_probs = self.measurement_model(x=None)
posterior_probs = self.state_prediction * self.probability #check this
#Normalize the probabilities to ensure they sum to 1
if np.sum(posterior_probs) > 0:
    self.probability = posterior_probs / np.sum(posterior_probs)
else:
    rospy.logwarn("Posterior probabilities sum to zero. Using uniform distribution.")
    self.probability = np.ones(self.num_states) / self.num_states
rospy.loginfo(f"Shape of state_prediction: {self.state_prediction.shape}")
rospy.loginfo(f"Updated belief: {self.probability}")

def state_update(self):
    rospy.loginfo("Updating state probabilities...")
    self.state_predict()
    measurement_probs = self.measurement_model(None)

    posterior_probs = np.zeros(self.num_states)
    for i in range(self.num_states):
        posterior_probs[i] = self.state_prediction[i] * measurement_probs[self.colour_map[i]]

    # Normalize
    total = np.sum(posterior_probs)
    if total != 1:
        self.probability = posterior_probs / total
    else:
        rospy.logwarn("Probabilities sum to zero. Using uniform distribution.")
        self.probability = np.ones(self.num_states) / self.num_states

    rospy.loginfo(f"Updated state probabilities: {self.probability}")

if __name__ == "__main__":

```

```

# This is the known map of offices by colour
# 0: red, 1: green, 2: blue, 3: yellow, 4: line
# current map starting at cell #2 and ending at cell #12
colour_map = [3, 0, 1, 2, 2, 0, 1, 2, 3, 0, 1]

target_offices = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12] #to be given by TA

# TODO calibrate these RGB values to recognize when you see a colour

colour_codes = [
    [230, 67, 114], # red
    [238, 90, 143], # red
    [230, 75, 117], # red
    [200, 162, 208], # p
    [198, 135, 178], # p
    [198, 141, 183], #p
    [251, 148, 94], #oranbge
    [248, 140, 89], #oranbge
    [250, 148, 100], #oranbge
    [217, 129, 108], #brown
    [217, 129, 110], #brown
    [217, 129, 110], #brown
    [150, 150, 150] #line
]

# initial probability of being at a given office is uniform
p0 = np.ones_like(colour_map) / len(colour_map)

localizer = BayesLoc(p0, colour_codes, colour_map)

rospy.init_node("final_project")
rospy.sleep(0.5)
rate = rospy.Rate(10)
twist = Twist()

"""
    TODO: complete this main loop by calling functions from BayesLoc, and
    adding your own high level and low level planning + control logic
"""

while not rospy.is_shutdown():
    try:
        rospy.loginfo("Following the line...")
        localizer.follow_line()

```

```

if localizer.cur_colour is not None:
    rospy.loginfo(f"Observed color (RGB): {localizer.cur_colour}")
    localizer.state_update()

    # Check if the current state matches a target office
    current_office = np.argmax(localizer.probability)

    if current_office == 12:
        rospy.loginfo(f"Observed color (RGB): {localizer.cur_colour}")
        localizer.state_update()
    elif current_office in target_offices:
        rospy.loginfo(f"Reached target office {current_office}. Stopping to deliver mail.")
        localizer.stop_at_office()
        target_offices.remove(current_office) # Remove delivered office
        localizer.state_update()

except Exception as e:
    rospy.logerr(f"Error: {e}")

rate.sleep()

rospy.loginfo("Finished!")
rospy.loginfo(f"Final state probabilities: {localizer.probability}")
rospy.loginfo(f"Final state probabilities: {np.argmax(localizer.probability)}")
rospy.loginfo(f"Final state probabilities: {np.sum(localizer.probability)}")

```